Memo – 7 Oefenvraestelle

Gebruik hierdie memo eers nadat jy die vraestel onder tydsdruk voltooi het. By Pomplemma-bewyse is die kern die formele struktuur; alternatiewe geldige keuses wat dieselfde teenstrydigheid bewys, kan aanvaarbaar wees.

Let wel: Die toetsinligting noem vier Pomplemma-bewyse, terwyl die notas vyf lys. Hierdie memo dek al vyf.

1. Oefenvraestel 1 – Gebalanseerde begin

Afdeling A

1.1 1.1 M=(Q,Σ,δ,q0,F); Q states, Σ alphabet, δ transition, q0 start, F accepting states.

1.2 1.2 Using dfa_end01 for 101101: q0→q1(1), q1→q0(0), q0→q1(1), q1→q2(1), q2→q1(0), q1→q2(1). Accept.

1.3 1.3 Three-state idea: q0=neither useful suffix, q1=last symbol is 0, q2=last two symbols are 01; q2 final. Equivalent transitions as shown.

1.4 1.4 Standard proof: assume regular; let p be pumping length; w=0^p1^p; w=xyz, |xy|≤p, |y|>0, so y=0^β, β≥1; choose k=2; xy^2z=0^(p+β)1^p, counts differ; contradiction; not regular.

Afdeling B

2.1 2.1 ^ anchors beginning; $ anchors end.

2.2 2.2 ^\d{8}$

2.3 2.3 ^(?=.*[A-Z])(?=.*[a-z])(?=.*\d).{8,}$

2.4 2.4 ^[A-Z]{3}-\d{4}$

2.5 2.5 ^[ab]*ab$

2.6 2.6 ^cat.*dog$

2.7 2.7 AB1234 G; A1234 O; GP0001 G; ab1234 O; ZZ9999 G; ABC1234 O.

Afdeling C

3.1 3.1 V variables, Σ terminals, S start variable, P productions.

3.2 3.2 Een geldige afleiding: S⇒0A⇒00A⇒001.

3.3 3.3 No: B→Ba is left-linear while the other rules are right-linear; mixing directions is outside the restricted regular-grammar form.

3.4 3.4 S→0S|1S|0A; A→1B; B→ε.

3.5 3.5 Q={S,A,B,f}, Σ={0,1}, q0=S, F={f}; S--0→A, S--1→f, A--1→B, A--0→A, B--ε→f.

3.6 3.6 CNF: S→DE|CA|BC|b; A→DA|a; B→BC|b; C→b; D→a; E→SC.

3.7 3.7 Ambiguous means one string has more than one parse tree/leftmost derivation. 2+3*4 can be 2+(3*4) or (2+3)*4.

2. Oefenvraestel 2 – Outomate & patrone

Afdeling A

1.1 1.1 DFA has exactly one next state for each state/symbol; NFA may have 0,1 or many. DFA has one path; NFA accepts if at least one path accepts.

1.2 1.2 For the even-1 DFA, 001011: q0→q0→q0→q1→q0→q1→q0. Accept.

1.3 1.3 The NFA can branch; for contains 01, 01 is accepted, while 11 is rejected.

1.4 1.4 Same formal proof as paper 1: assume regular, w=0^p1^p, y=0^β, choose k=2, get 0^(p+β)1^p∉B, contradiction.

Afdeling B

2.1 2.1 *=0+, +=1+, ?=0 or 1.

2.2 2.2 ^0\d{9}$

2.3 2.3 ^(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/\d{4}$

2.4 2.4 ^([01][0-9]|2[0-3]):[0-5][0-9]$

2.5 2.5 ^#[A-Fa-f0-9]{6}$

2.6 2.6 ^[a-z]{5,10}$

2.7 2.7 \d+ can match a substring; ^\d+$ anchors the whole string.

Afdeling C

3.1 3.1 Examples: S⇒a; S⇒Sb⇒ab; S⇒A⇒b. Any two with full derivation earn.

3.2 3.2 Terminals are final symbols; variables/non-terminals can still be replaced.

3.3 3.3 NFA: S--a→A, S--b→f; A--a→S, A--ε→f. Start S, final f.

3.4 3.4 P→ε|0|1|0P0|1P1.

3.5 3.5 P⇒0P0⇒01P10⇒0110.

3.6 3.6 One acceptable CNF route: introduce E→(and F→), then binarise S→SS and S→E S F. Remove ε from S carefully; because S is start, preserve S→ε if ε is in the language, often using a fresh start S0. A fully correct answer may use S0 and then binarise. Core final forms must be A→BC or A→a (plus S0→ε if allowed).

3.7 3.7 Mixing left/right linear forms can enforce matching counts, e.g. {0^n1^n}, which is not regular.

3. Oefenvraestel 3 – Regex intensief

Afdeling A

1.1 1.1 |y|>0 prevents pumping an empty part; |xy|≤p forces y into the first p symbols.

1.2 1.2 Even-1 DFA on 110101: q0→q1→q0→q1→q0→q1→q0; accept.

1.3 1.3 q0=even 0s, q1=odd 0s; 0 toggles, 1 loops; q1 final.

1.4 1.4 Assume regular; p; w=0^(p+1)1^p; y=0^β, β≥1; choose k=0; new counts p+1−β and p. If β=1 equal; if β>1 fewer 0s. In neither case n>m; contradiction.

Afdeling B

2.1 2.1 [^0-9] any non-digit; \D same; \W non-word; \S non-whitespace.

2.2 2.2 ^user_\d{4}$

2.3 2.3 ^.+\.txt$

2.4 2.4 ^[A-Z]{3}-\d{3}$

2.5 2.5 ^[01]*1[01]*$

2.6 2.6 ^-?\d+\.\d+$

2.7 2.7 cat and dog match; catdog, Cat, dog<space> do not; dog repeated in table is G.

Afdeling C

3.1 3.1 S→0S|1S|0A; A→1B; B→ε.

3.2 3.2 Same NFA conversion: S--0→A, S--1→f; A--1→B, A--0→A; B--ε→f.

3.3 3.3 KVG has unrestricted strings of terminals/non-terminals on RHS and supports recursion; regular grammar is restricted to forms such as A→aB, A→a, A→B, A→ε.

3.4 3.4 E⇒E+T⇒T+T⇒F+T⇒a+T⇒a+T*F⇒a+F*F⇒a+b*F⇒a+b*c.

3.5 3.5 Two structures: 2+(3*4) and (2+3)*4.

3.6 3.6 CNF: S→EB|DA; A→AC|DF; B→DS|EG; C→a? Wait terminals: for a use E, b use D, s use C. Thus C→s, D→b, E→a; A→AC|DF where F→AA; B→DS|EG where G→BB. Final: S→EB|DA; A→AC|DF; B→DS|EG; C→s; D→b; E→a; F→AA; G→BB.

4. Oefenvraestel 4 – NFA/DFA & grammatika

Afdeling A

1.1 1.1 ε moves without consuming an input symbol.

1.2 1.2 ε-closure(q0)={q0,q1} for the shown sketch.

1.3 1.3 Partition final/non-final, mark distinguishable pairs, refine groups, merge equivalent states. Final and non-final cannot be equivalent.

1.4 1.4 Assume regular; p; choose w=0^p1 0^p1=(0^p1)(0^p1); y=0^β; choose k=2; first half changes but second does not, so halves are not identical; contradiction.

Afdeling B

2.1 2.1 (ab)+ repeats the whole block; ab+ means a followed by one or more b's.

2.2 2.2 ^ID\d{6}$

2.3 2.3 ^[AEIOUaeiou]{2}\d{3}$

2.4 2.4 ^https://.*$

2.5 2.5 ^[A-Za-z]+$

2.6 2.6 ^[A-Z][a-z]+([A-Z][a-z]+)*$

2.7 2.7 \ escapes metacharacters; literal dot = \.

2.8 2.8 ^123 ?456 ?7890$ is acceptable for optional single spaces; if only the two exact formats are required, ^(1234567890|123 456 7890)$.

Afdeling C

3.1 3.1 S→b gives b; S→aS→ab; S→aS→aab. Language a*b.

3.2 3.2 S→0S|1S|0A; A→1B; B→ε.

3.3 3.3 Q={S,A,f}, Σ={a,b}, q0=S, F={f}; S--a→A, S--b→f; A--a→S, A--ε→f.

3.4 3.4 P→ε|00|11|0P0|1P1.

3.5 3.5 P⇒1P1⇒10P01⇒1001.

3.6 3.6 ε-productions → unit productions → terminal replacements → binarise long productions.

3.7 3.7 S→ABC is not CNF because RHS has three variables; introduce D→BC and S→AD.

3.8 3.8 Regular grammar is restricted; KVG allows arbitrary RHS strings of terminals/non-terminals and recursion.

5. Oefenvraestel 5 – Besluitbaarheid & CNF

Afdeling A

1.1 1.1 Membership asks whether a specific string is in L; emptiness asks whether L contains any string.

1.2 1.2 1010110 on even-1 DFA: q0→q1→q0→q1→q0→q1→q0→q0; accept.

1.3 1.3 For a complete DFA, swap accepting and non-accepting states.

1.4 1.4 Assume regular; choose prime q>p; w=0^q; y=0^β; choose k=q+1; new length q+qβ=q(β+1), a composite number; contradiction.

Afdeling B

2.1 2.1 . = one arbitrary character; .* = zero or more arbitrary characters.

2.2 2.2 ^[A-Z]{3}\d{4}$

2.3 2.3 ^(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/\d{4}$

2.4 2.4 ^([01][0-9]|2[0-3]):[0-5][0-9]$

2.5 2.5 ^(?=.*[A-Z])(?=.*[a-z])(?=.*\d).{8,}$

2.6 2.6 ^0\d{9}$

2.7 2.7 {n}=exactly n; {n,}=at least n; {n,m}=between n and m.

Afdeling C

3.1 3.1 G=(V,Σ,S,P).

3.2 3.2 S→aS|bS|b is one valid right-linear grammar for all nonempty strings ending in b.

3.3 3.3 A→Ba is left-linear while A→aB is right-linear; mixing can go beyond regular grammars.

3.4 3.4 S→a after removing units.

3.5 3.5 C→a, D→b; S→CB|DA; A→CA|a; B→DB|b.

3.6 3.6 Root=start variable; internal nodes=variables; leaves=terminals; left-to-right leaves=yield/string.

3.7 3.7 E→E+E|E*E|getal is ambiguous because both 2+(3*4) and (2+3)*4 are derivable.

3.8 3.8 A→BC and A→a (plus start→ε special case).

6. Oefenvraestel 6 – Gemengde eksamensimulasie

Afdeling A

1.1 1.1 M=(Q,Σ,δ,q0,F); δ maps a state and input symbol to exactly one next state in a DFA.

1.2 1.2 begin-with-1 DFA on 101001: q0→q1→q1→q1→q1→q0→qd. Reject.

1.3 1.3 NFA accepts if at least one complete path ends in a final state; DFA has exactly one path.

1.4 1.4 Standard 0^p1^p proof; k=2 gives 0^(p+β)1^p, not equal counts.

Afdeling B

2.1 2.1 ^0\d{9}$

2.2 2.2 One acceptable simplified class-notes regex: ^[^.](?!.*\.\.)[A-Za-z0-9._%+-]*[^.]@[A-Za-z0-9-]+\.[A-Za-z]{2,}(\.[A-Za-z]{2,})?$.

2.3 2.3 ^[A-Z]{3}-\d{4}$

2.4 2.4 ^\d+$

2.5 2.5 ^[01]*01[01]*$

2.6 2.6 ^[a-z]{5,10}$

2.7 2.7 ^ = start, cat literal, .* any chars 0+, dog literal, $ = end.

Afdeling C

3.1 3.1 S⇒Ab⇒Aab⇒aab.

3.2 3.2 S→0S|1S|0A; A→1B; B→ε.

3.3 3.3 S--a→A; A--b→B; B--ε→f; q0=S; F={f}.

3.4 3.4 P→ε|0|1|0P0|1P1; P⇒0P0⇒01P10⇒0110.

3.5 3.5 S→ABC is not binary. Use D→BC; S→AD; A→a; B→b; C→c.

3.6 3.6 ε productions first; unit productions second; terminal replacement third; binarisation fourth.

3.7 3.7 Inherent ambiguity means the language itself cannot be generated by any unambiguous CFG.

7. Oefenvraestel 7 – Finale 75-punt oefentoets

Afdeling A

1.1 1.1 Regex: validation/search/pattern matching. Finite automata: lexical analysis, protocol/input-state validation, pattern recognition.

1.2 1.2 001010 on odd-0 DFA: q0→q1→q0→q1→q1→q0→q1; accept.

1.3 1.3 Start with {q0}; for each subset and symbol take union of destinations; repeat new subsets; subsets containing NFA final states are final.

1.4 1.4 Assume regular; p; w=0^(p+1)1^p; y=0^β; choose k=0; n=p+1−β, m=p; β=1 gives n=m, β>1 gives n<m; contradiction.

Afdeling B

2.1 2.1 ^\d{8}$

2.2 2.2 ^#[A-Fa-f0-9]{6}$

2.3 2.3 ^([01][0-9]|2[0-3]):[0-5][0-9]$

2.4 2.4 ^[01]*01$

2.5 2.5 .*\d.*

2.6 2.6 ^[A-Z]{3}-\d{3}$

2.7 2.7 ^ start; $ end; [] class; () group; | OR; \ escape.

Afdeling C

3.1 3.1 G=(V,Σ,S,P); derive by starting at S and replacing variables until only terminals remain.

3.2 3.2 S→0S|1S|0A; A→1B; B→ε.

3.3 3.3 Q={S,A,f}, Σ={a,b}, q0=S, F={f}; S--a→A, S--b→f; A--a→S, A--ε→f.

3.4 3.4 P→ε|0|1|0P0|1P1; P⇒1P1⇒10P01⇒1001.

3.5 3.5 Ambiguous grammar gives both 2+(3*4) and (2+3)*4.

3.6 3.6 Same CNF as source-style conversion: C→b, D→a; S→DE|CA|BC|b; A→DA|a; B→BC|b; E→SC.

3.7 3.7 A→BC and A→a; start variable may additionally produce ε where allowed.